

Resumen Final de SOLID:

S (SRP): Cada clase hace solo una cosa.

O (OCP): Usar herencia o interfaces en lugar de ifs eternos.

L (LSP): Una clase hija no debe romper el comportamiento de su padre.

I (ISP): Dividir interfaces grandes en interfaces pequeñas y específicas.

D (DIP): No depender de clases concretas, sino de interfaces o abstracciones.

S - Single Responsibility Principle (SRP)

Cada clase debe hacer solo una cosa.

✗ Incorrecto: Una clase hace más de una cosa.

```
class Car {  
    public void drive() {  
        System.out.println("El auto está en movimiento...");  
    }  
  
    public void saveToDatabase() {  
        System.out.println("Guardando el auto en la base de  
datos...");  
    }  
}
```

➡ Problema: `Car` maneja tanto la lógica de conducción como la de guardar en la base de datos.

✓ Correcto: Separar responsabilidades en clases distintas.

```
class Car {  
    public void drive() {  
        System.out.println("El auto está en movimiento...");  
    }  
}  
  
class CarDatabase {  
    public void saveToDatabase(Car car) {
```

```
        System.out.println("Guardando el auto en la base de
datos...");
    }
}
```

Ahora **Car** solo maneja la conducción, y **CarDatabase** solo maneja la base de datos.

O - Open/Closed Principle (OCP)

Cada vez que agregamos algo nuevo modificamos la clase y no debería ser así **Usar herencia o interfaces en lugar de ifs eternos.**

✗ Incorrecto: Un **if con todos los tipos de vehículos.**

```
class Vehicle {
    public void printType(String type) {
        if (type.equals("Car")) {
            System.out.println("Es un auto.");
        } else if (type.equals("Bike")) {
            System.out.println("Es una bicicleta.");
        }
    }
}
```

⊖ Problema: Si agregamos una moto, tenemos que modificar **Vehicle**.

✓ Correcto: Usamos herencia y polimorfismo.

```
abstract class Vehicle {
    abstract void printType();
}

class Car extends Vehicle {
    public void printType() {
        System.out.println("Es un auto.");
    }
}

class Bike extends Vehicle {
```

```
    public void printType() {  
        System.out.println("Es una bicicleta.");  
    }  
}
```

✓ Ahora podemos agregar más vehículos sin modificar `Vehicle`.

3L - Liskov Substitution Principle (LSP)

Una clase hija no debe romper el comportamiento de su padre.

✗ Incorrecto: Un hijo tiene un método que no puede usar.

```
class Vehicle {  
    public void startEngine() {  
        System.out.println("Encendiendo motor...");  
    }  
}  
  
class Bicycle extends Vehicle { } // ¡Error! No tiene motor.
```

➡ Problema: Si intentamos `bicycle.startEngine()`, el código fallará.

✓ Correcto: Separar vehículos con y sin motor.

```
abstract class Vehicle { }  
  
abstract class MotorVehicle extends Vehicle {  
    abstract void startEngine();  
}  
  
class Car extends MotorVehicle {  
    public void startEngine() {  
        System.out.println("El auto está encendiendo...");  
    }  
}  
  
class Bicycle extends Vehicle { } // No tiene un motor que no puede  
usar.
```

✓ Ahora **Bicycle** no tiene un método que no necesita.

I - Interface Segregation Principle (ISP)

Dividir interfaces grandes en interfaces pequeñas y específicas.

✗ **Incorrecto: Una interfaz obliga a implementar métodos innecesarios.**

```
interface Vehicle {
    void startEngine();
    void pedal_bici();
}

class Car implements Vehicle {
    public void startEngine() {
        System.out.println("El auto está encendiendo...");
    }

    public void pedal_bici() {
        throw new UnsupportedOperationException("Un auto no tiene pedales de bici!");
    }
}
```

⊖ **Problema:** **Car** no debería implementar **pedal()**.

✓ **Correcto: Separar en interfaces específicas.**

```
interface EngineVehicle {
    void startEngine();
}

interface PedalVehicle {
    void pedal_bici();
}

class Car implements EngineVehicle {
    public void startEngine() {
        System.out.println("El auto está encendiendo...");
    }
}
```

```

    }
}

class Bicycle implements PedalVehicle {
    public void pedal_bici() {
        System.out.println("Pedaleando la bicicleta...");
    }
}

```

✅ Ahora cada clase **solo implementa lo que necesita**.

D - Dependency Inversion Principle (DIP)

No depender de clases concretas (sobre todo pequeñas), sino de interfaces o abstracciones.

❌ **Incorrecto: Car depende directamente de ElectricEngine.**

```

class ElectricEngine {
    public void start() {
        System.out.println("Motor eléctrico encendido...");
    }
}

class Car {
    private ElectricEngine engine;

    public Car() {
        this.engine = new ElectricEngine();
    }

    public void start() {
        engine.start();
    }
}

```

❌ **Problema:** Si queremos cambiar a un motor de gasolina, tenemos que modificar **Car**.

✅ **Correcto:** Usamos una interfaz para desacoplar **Car** del motor.

```

interface Engine {
    void start();
}

class ElectricEngine implements Engine {
    public void start() {
        System.out.println("Motor eléctrico encendido...");
    }
}

class GasolineEngine implements Engine {
    public void start() {
        System.out.println("Motor de gasolina encendido...");
    }
}

class Car {
    private Engine engine;

    public Car(Engine engine) { // Recibe el motor desde afuera
        this.engine = engine;
    }

    public void start() {
        engine.start();
    }
}

```

Ahora podemos hacer:

```

Car electricCar = new Car(new ElectricEngine());
electricCar.start(); // Motor eléctrico encendido...

```

```

Car gasolineCar = new Car(new GasolineEngine());
gasolineCar.start(); // Motor de gasolina encendido...

```

✅ **Car ya no depende de un motor específico**, lo hace más flexible y fácil de cambiar.

```
class MySQLDatabase {  
    void connect() {  
        System.out.println("Conectando a MySQL...");  
    }  
}
```

```
class App {  
    private MySQLDatabase database;  
  
    public App() {  
        this.database = new MySQLDatabase(); // DEPENDENCIA DIRECTA  
    }  
  
    void start() {  
        database.connect();  
    }  
}
```

// Interfaz (Abstracción)

```
interface Database {  
    void connect();  
}
```

// Implementaciones concretas

```
class MySQLDatabase implements Database {
```

```
public void connect() {  
    System.out.println("Conectando a MySQL...");  
}  
}
```

```
class PostgreSQLDatabase implements Database {  
    public void connect() {  
        System.out.println("Conectando a PostgreSQL...");  
    }  
}
```

// La aplicación ahora depende de una abstracción (Database)

```
class App {  
    private Database database;  
  
    public App(Database database) {  
        this.database = database;  
    }  
  
    void start() {  
        database.connect();  
    }  
}
```

// Uso

```
public class Main {
```

```
public static void main(String[] args) {  
    Database db = new MySQLDatabase(); // Se puede cambiar fácilmente a PostgreSQL  
    App app = new App(db);  
    app.start();  
}  
}
```